

Chapter 7

Empowering Scientific Computing and Data Manipulation With Numerical Python (NumPy)

Tesfaye Fufa Gedefa

 <https://orcid.org/0000-0001-5405-5320>

Space Science and Geospatial Institute, Ethiopia

Galety Mohammed Gouse

 <https://orcid.org/0000-0003-1666-2001>

Samarkand International University of Technology, Uzbekistan

Garamu Tilahun Iticha

Debre Markos University, Ethiopia

ABSTRACT

NumPy is a Python library for performing numerical data structures and specialized computing. It improves the components of N-dimensional arrays and provides operations and tools to interface with these arrays. NumPy implements the N-dimensional array, or ndarray, and provides Python-specific scientific methods for performing realistic array and matrix operations. When compared to array programming in other languages, it will allow us to do a wide range of mathematical operations and data manipulations. NumPy can be used with other Python packages and programming languages such as C and C++. NumPy now supports object-oriented programming as well. For example, a class called ndarray may be an N-dimensional array with multiple ways of performing various data structure operations and characteristics.

DOI: 10.4018/978-1-6684-7100-5.ch007

INTRODUCTION

NumPy is an essential package for systematic computing in Python. The library gives an N-dimensional array question and different inferred objects (such as masked arrays and matrices) (NumPy Developers, 2008-2022). It is a mixture of sequences for fast procedures on arrays, with mathematical, logical, and shape manipulation; sorting; selecting; input/output; discrete Fourier transforms; introductory linear algebra; basic statistical techniques; arbitrary simulation; and more (Johansson, 2019). NumPy delivers the numerical backend for nearly every scientific or technical library for Python. It provides an N-dimensional array object, a robust data structure that has become Python's standard numerical data representation (Millman, 2011). It is, therefore, a vital part of the scientific Python ecosystem (Johansson, 2019). Its goal is to create the cornerstone of a suitable environment for scientific computing. As stated (Agathiya Raja, 2022), the Python library plays a significant role in different application areas for analyzing complex networks, data science, and big data; analyzing and visualizing the networks using Python offers good insights about the networks to end-users using the python library. To better understand the people surrounding NumPy, at the core of the NumPy package is the ndarray object (Alammar, 2022).

This encapsulates n-dimensional arrays of the same data types, with many operations performed in the accumulated code for performance. NumPy is an essential bundle for scientific computing in Python, and it provides an n-dimensional array object and numerous derived things (such as screened arrays and matrices). The ndarray object is at the heart of the NumPy bundle. However, NumPy arrays and standard Python sequences differ in several ways (Johansson, 2019; Albenis Pérez-Alarcón, 2021).

Python extends Python with robust mathematical operations that provide practical guarantees in calculations with arrays and matrices. NumPy stores a massive library of high-level mathematical functions that work on these arrays and matrices. NumPy aims to deliver an array object faster than traditional Python lists. As a result, Python is the fastest to read fewer bytes of memory or has contiguous memory. It assists with different operations like slicing, indexing, and broadcasting; matrix computational functions; and has the benefits of single instruction multiple data vector processes (SIMD). In addition, NumPy completely supports an object-oriented approach, and it is an N-dimensional array named ndarray, a class possessing various techniques and qualities. Functions mirror several methods in the outermost NumPy namespace. This lets the languages code in whatever makes the most sense.

Empowering Scientific Computing and Data Manipulation

Installing NumPy

This chapter strongly recommends using a technical Python source to install NumPy. In this chapter, looking for the complete commands to mount NumPy on the operating system, see Connecting NumPy and if there is already installed Python, easily connect NumPy with the following:

```
pip install NumPy, or > conda install NumPy
```

Importing NumPy

Importing Modules: To use the NumPy library in this chapter, import it into the program. The NumPy module is often imported using the alias np. NumPy should be imported as np.

```
import NumPy as np
```

After this, this chapter can access functions and classes in the NumPy module using the np namespace. It shortens the imported name to np for better readability (alias) of the code using NumPy. This is a widely adopted convention, so anyone working with code can easily understand it.

NUMPY DATATYPES

The most common way to manipulate values in NumPy is through ndarray objects (Johansson, 2019). Because ndarrays can hold any number of quantities, they can represent any data type, such as scalars, vectors, matrices, or tensors. They are similar to Python lists but can have multiple dimensions. The basic numeric information types saved by NumPy are listed in Figure 1. Non-numerical data sortings, such as text, objects, and custom compound sorts, are also accessible. Datasets for Basic Numeric Data that may be accessed using NumPy Scalar: A scalar is a single integer with zero dimension, and NumPy has one. As a result, instead of Python int, NumPy data types utilize collations such as uint8, int8, uint16, and int16. Strings, objects, and complicated data types the user creates are also supported.

Figure 1. Basic numerical data types NumPy
Source: NumPy Developers (2008-2022)

dtype	Variants	Description
int	int8, int16, int32, int64	Integers
uint	uint8, uint16, uint32, uint64	Unsigned (nonnegative) integers
bool	Bool	Boolean (True or False)
float	float16, float32, float64, float128	Floating-point numbers
complex	complex64, complex128, complex256	Complex-valued floating-point numbers

Scalar

Scalar and NumPy are terms for single numbers with a dimension of zero. As a result, this book chapter may pass the value to NumPy’s array function to generate a NumPy array that holds a scalar instead of Python’s int types, such as uint8, int8, uint16, and int16. It returns an empty parenthesis to indicate that it has no dimensions (Mulani, 2022)

Vectors

It is a one-dimensional list of numbers. To create a vector, pass a Python list to the array function (Mulani, 2022).

Matrices

In Numpy, matrices are represented as lists of lists or 2D vectors. Like vectors, this chapter generates matrices using NumPy’s array function. However, one must provide a list where each A list denotes a row rather than merely passing through a single list. Consequently, to make a three-by-three matrix using digits 1 through 9 (McKinney, 2012).

Tensors

N-dimensional ndarrays are tensors. They can include additional dimensions similar to vectors (Alammar, 2022).

*Empowering Scientific Computing and Data Manipulation***NUMPY ARRAYS**

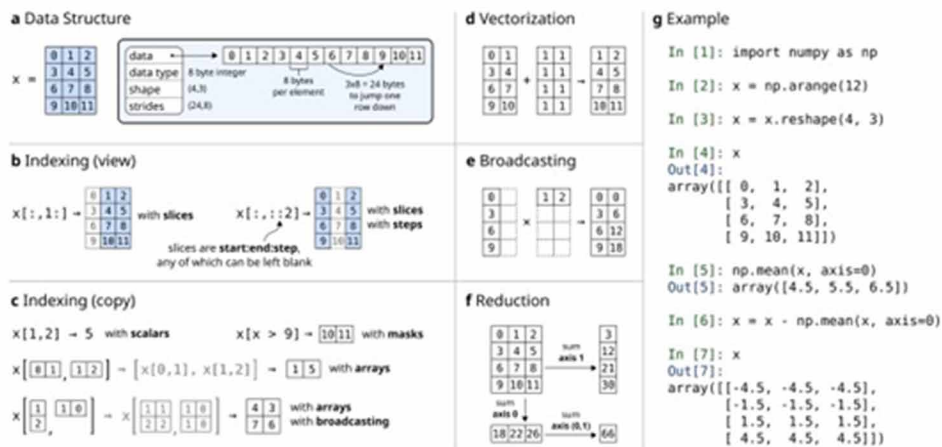
A data structure that effectively stores and accesses multidimensional arrays is the NumPy array. A pointer to memory is included in Numpy, along with metadata used to convert the data stored there—particularly information sorting, shaping, and steps. The data sort shows the kinds of parts stored in an array. The information sorting process depicts the personalities of components organized into an array. A data sort is included in an array, and the memory requirements for each cluster element are the same (Harris, 16 September 2020). In Python, NumPy arrays are the standard way to store numerical data and allow numerical operations to be done in a high-level language. As stated by (Oliphant, 2006), an array is a form of data that stores multiple values using just two identifiers.

An array is a collection of data pieces that can be accessed by their directories and are organized sequentially. Lists of numerical data, vectors, and matrices are stored in NumPy arrays. The NumPy library has numerous procedures (built-in functions) for creating, modifying, and modifying NumPy arrays. Even though the Python programming language has an array data structure, it is less flexible, practical, and powerful than the NumPy array. The NumPy Connect memory location: Each position is only assigned to one piece of data, and the memory space needs to be partitioned into precisely sized areas (Stefan van der Walt, 2021). Contiguous memory allocation is now available: To ensure that memory space can be used, split the data into pieces and put them in different places.

Figure 2 shows that the metadata areas are connected to the NumPy array data structure. Ordering an array using cuts is what b is about. These forms give back a summary of the initial data. c. arrange an array using masks, scalar aids, or other clusters in a way that duplicates the initial data. An array is ordered with different arrays in the bottom example, broadcasting the ordering disputes before the lookup of the original data. The bottom example discloses the indexing arguments before the lookup by indexing an array with several arrays. d. The use of vectorization allows operations to be applied to groups of elements. e. As two-dimensional arrays increase, so does broadcasting. f, acting in conjunction with one or more tomahawks while reducing operations. In this scenario, an array is added to a few tomahawks to produce a vector continuously or two tomahawks to produce a scalar. Using two consecutive axes to construct a scalar is explained in g, an illustration of NumPy code. An example, the NumPy code in g, demonstrates some of these ideas.

Empowering Scientific Computing and Data Manipulation

Figure 2. The NumPy array incorporates various fundamental features of the Array
Source: Harris (2020)



THE DIFFERENCE BETWEEN A NUMPY ARRAY AND A LIST

There are three benefits to using a Python NumPy array instead of a list. Less memory, faster, and more convenient. The Python NumPy array should be the first option because it uses less memory than a list. Then, it is relatively quick to implement and works well with NumPy simultaneously.

Figure 3. The difference between a NumPy array and a list
Source: Alammr (2022)

List:



Numpy:



In Figure 3, the list and Numpy array store data in memory. However, there is a significant difference. Performing these operations on a list can be tedious and

Empowering Scientific Computing and Data Manipulation

time-consuming when searching, updating, and manipulating data from memory. On the other hand, using a NumPy array is advantageous as it allows for efficient data manipulation directly from memory due to its sequential data storage.

Difference between List and ArrayTable

- **Changeability:** An array can keep the features of several different data types, but it can only store the basics of one data type.
- **Data Types Storage:** The list can store various data, while the Array provides equivalent data values.
- **Importing Packages:** Bringing in parcels No container is necessary before utilizing a list because the Python programming language is an inbuilt data structure. The Array, however, is not a feature of the NumPy language. Therefore, we must ingest the “collection” component before creating and utilizing arrays.
- **Arithmetical Process:** The NumPy module allows an array assembly to save data values and modify them quickly, which makes using an array advantageous during mathematical operations in Python. On the other hand, the outcomes are not shown in the list. Due to this, math operations with the list are still possible but less practical than those with the collection.
- **Alter Competences:** The list, on the other hand, is an in-built data construction and hence can be resized and adjusted very rapidly and efficiently, whereas the Array consumes very little memory, changing the memory position.

ARRAY OPERATIONS

When an array is declared, we can access its elements or carry out particular operations. The last session taught us how to access items. The various functions that can be used with arrays are covered in this section.

Arithmetic Operations

Mathematical operations on NumPy arrays are quick and straightforward. An essential arithmetic operation, such as addition, subtraction, multiplication, division, etc., is performed on each pair of components when applied to two collections. For example, combine two arrays, the first element from the first Array will be added to the first element of the second Array, and so on. Consider the subsequent element-wise operations on two arrays:

Array Transpose

To reverse the tensor swap using the keyword argument, use transpose (an argsort (axes)). Transposing a one-dimensional array produces an unchanged copy of the original Array.

Array Sorting

This involves pushing components in logical order. Any sequence with an element-based order, such as numeric or alphabetical, ascending or descending, is considered well-ordered. For example, a function called sort () on the NumPy n-dimensional array element sorts an array.

Concatenating Arrays and Splitting

Every code that came before it only operated on individual arrays. On the other hand, it is also possible to join many arrays into a single array, and it is also possible to break a single array into several arrays. The concatenation process refers to joining two or more arrays together. When arrays are concatenated, sequences are added one after the other in sequential order.

Array Reshaping

Reshaping entails changing an array's shape. The number of necessary elements in each length determines the form of an array. By reshaping, we can change the size of the Array, the number of elements in each, or both.

As shown in Figure 4, reshaping the Array with the help of NumPy to change the dimension as we want using rows and columns by changing the size of the Array.

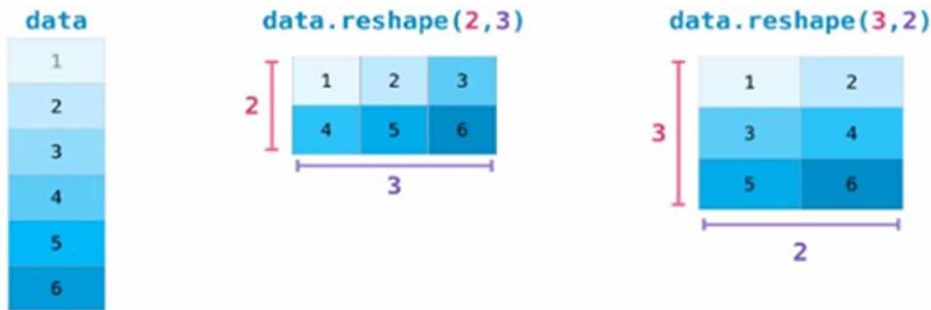
Slicing and Indexing

This approach makes it possible to choose any item in an array depending on its N-dimensional index, allowing for more flexibility in the selection process. Every integer array has a representation of the total number of indexes for its respective dimension. When the index is composed of the same number of integer arrays as the dimensions of the ndarray being targeted, completing the operation becomes quite simple. NumPy uses the Python language's sophisticated indexing (McKinney, 2012) capability. Python's indexing features were initially developed in response to the demands of users who worked with numbers. Slicing is removing a chosen

Empowering Scientific Computing and Data Manipulation

Figure 4. Data reshaping

Source: Java Tutorial Developers (2022)



group of components from an array. This division process is comparable to the one used in the list. Think about the following:

Figure 5. Slicing a NumPy array

Source: Oliphant (2006)

```
>>> a[0, 3:5]
array([3, 4])

>>> a[4:, 4:]
array([[44, 55],
       [54, 55]])

>>> a[:, 2]
a([2, 12, 22, 32, 42, 52])

>>> a[2::2, ::2]
array([[20, 22, 24],
       [40, 42, 44]])
```

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

As described in Figure 5, the slicing of an array in NumPy is the way of removing a chosen group of components from a given an array.

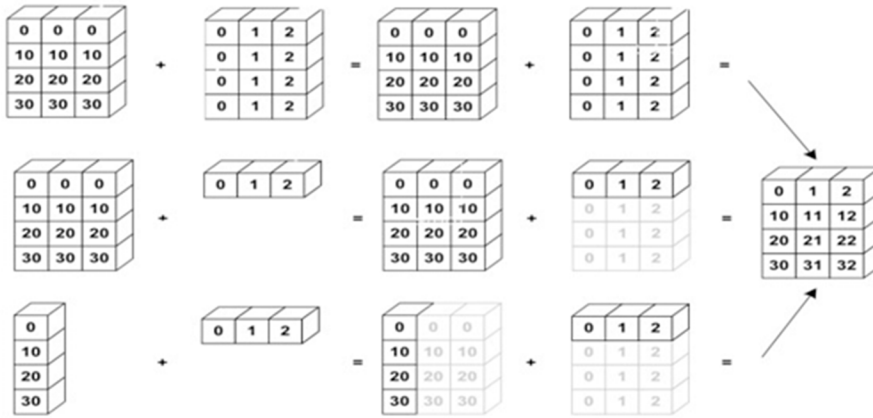
Broadcasting

The ability of NumPy to handle arrays with varying dimensions appropriately while performing arithmetic operations is referred to as broadcasting. Arithmetic operations are often performed on items corresponding to one another when working with arrays. These procedures may be carried out without a hitch so long as the two

Empowering Scientific Computing and Data Manipulation

Figure 6. Numpy array broadcasting

Source: McKinney (2012)



arrays being worked with have an identical structure. Vital processes on NumPy arrays are element-based. This works on arrays of the same degree. In any case, it is imaginable to perform actions on arrays of various sizes if NumPy can change them; this change is called broadcasting. Figure 6 gives an example of broadcasting.

In any case, it is also imaginable to perform actions on arrays of various sizes on the off chance that NumPy can change them/

SPECIAL FUNCTION OF NUMPY PYTHON

Numerous numerical capabilities in NumPy allow for a variety of numerical operations. Trigonometric aptitudes, arithmetic skills, and the capacity for working with complex numbers are all examples of numerical aptitudes (NumPy Developers, 2008-2022). Let us talk about the mathematical functions first, in any case.

Trigonometric Functions

Figure 7 shows that the trigonometric formulas for computing the sine, cosine, and tangents of various angles in radians are available in NumPy. The sin, cos, and tan functions return the trigonometric ratio for the supplied angles.

Empowering Scientific Computing and Data Manipulation

Figure 7. Trigonometric functions
Source: Mulani (2022)

```
import numpy as np
arr = np.array([0, 30, 60, 90, 120, 150, 180])
print("\nThe sin value of the angles",end = " ")
print(np.sin(arr * np.pi/180))
print("\nThe cosine value of the angles",end = " ")
print(np.cos(arr * np.pi/180))
print("\nThe tangent value of the angles",end = " ")
print(np.tan(arr * np.pi/180))
```

Rounding Functions

NumPy offers a variety of routines for truncating the value of a decimal number tuned to the accuracy of decimal digits. First, let us discuss the adjusting features—the numpy. The `around ()` method works as intended. This task returns the corrected decimal value at the desired decimal location. Figure 8 contains the work’s syntax.

Figure 8. Numpy rounding functions

SN	Parameter	Description
1	num	It is the input number.
2	decimals	It is the number of decimals which to which the number is to be rounded. The default value is 0. If this value is negative, then the decimal will be moved to the left.

Statistical Functions in NumPy

As shown in Figure 9, Numerous statistical functions offered by Numpy can be utilized to analyze statistical data. For instance, identifying the Array’s most minor

Empowering Scientific Computing and Data Manipulation

Figure 9. Numpy statistical functions for maximum and minimum
Source: Millman (2011)

```
import numpy as np

a = np.array([[2,10,20],[80,43,31],[22,43,10]])

print("The original array:\n")
print(a)

print("\nThe minimum element among the array:",np.amin(a))
print("The maximum element among the array:",np.amax(a))

print("\nThe minimum element among the rows of array",np.amin(a,0))
print("The maximum element among the rows of array",np.amax(a,0))

print("\nThe minimum element among the columns of array",np.amin(a,1))
print("The maximum element among the columns of array",np.amax(a,1))
```

and extreme elements using NumPy.amin() and NumPy.amax() can find each array element’s minimum and maximum values along the required axis.

NumPy Sorting and Searching

Numerous sorting and searching functions are available in Numpy. The sort () method is used to build several sorting algorithms, including quicksort, merge sort, and heapsort. The function call must specify the sorting algorithm utilized in the

Figure 10. NumPy sorting and search algorithms

SN	Algorithm	Worst case complexity
1	Quick Sort	$O(n^2)$
2	Merge Sort	$O(n \cdot \log(n))$
3	Heap Sort	$O(n \cdot \log(n))$

Empowering Scientific Computing and Data Manipulation

sort operation. Let us talk about the sorting algorithm used by NumPy. Sort () is given in Figure 10.

Linear Algebra in NumPy

NumPy gives various capabilities to perform multiple algebraic calculations on the input data. Numpy linear algebra is shown in Figure 11.

Figure 11. NumPy linear algebra

SN	Function	Definition
1	dot()	It is used to calculate the dot product of two arrays.
2	vdot()	It is used to calculate the dot product of two vectors.
3	inner()	It is used to calculate the inner product of two arrays.
4	matmul()	It is used to calculate the matrix multiplication of two arrays.
5	det()	It is used to calculate the determinant of a matrix.
6	solve()	It is used to solve the linear matrix equation.
7	inv()	It is used to calculate the multiplicative inverse of the matrix.

Numpy Copies and Views

While executing the functions, some return a copy of the input array, while some return the view. When the contents are physically stored in another location, it is called Copy. If, on the other hand, a different view of the same memory content is provided, we call it a View (NumPy Developers, 2008-2022). A replica of the array object is not created by simply assigning values to its elements. Instead, it accesses it using the same id () used for the first Array. The id () method in Python is analogous to the pointer function in C in that it returns an object’s universal identification. In addition, any changes that occur in one are immediately reflected in the other. For example, if one changes its form, this will cause the other to alter its shape as well.

CONCLUSION

Most of this chapter is devoted to examining the importance of NumPy and the applications that may use its various packages. When using Python for technical and scientific computing, grasping the NumPy package ecosystem is essential. How the packages are employed for computing is necessary to get the NumPy package ecosystem and how they are employed. This is because of the nature of the computing tasks that Python is used for. It was underlined that matrices, vectors, ndarray, tensors, reshaping array expressions, and arithmetic operators should all be employed to perform a realistic array analysis. This chapter briefly introduces the NumPy library, emphasizing its mathematical operations and benefits when dealing with data structures. NumPy is a library that is important for doing computations in Python, and it has the potential to serve as the basis for all other Python computational libraries. The Python Scientific Community first developed NumPy. The ndarray, which is NumPy's data structure for N-dimensional arrays, was the topic of our discussion at the outset of this subsection. Then, we looked at how the ndarray may be used to generate and manage arrays, count orders, solve issues with data structures, carry out mathematical calculations, design applications for data science, and clip off sections of arrays.

NumPy can be used with other Python packages and programming languages such as C and C++. In addition, NumPy now supports object-oriented programming as well. For example, a class called ndarray may be an N-dimensional array with multiple ways of performing various data structure operations and characteristics. In conclusion, NumPy gives users access to various functions that can execute basic mathematical operations. Users can design applications based on data structures, generate arrays with N dimensions with the aid of this package, and adjust the geometry of existing arrays. Users must consider every detail seriously to get any use out of computers.

REFERENCES

- Agathiya Raja, G. K. (2022). *Introduction To Python for Social Network Analysis*. Wiley Online Library.
- Alammar, J. (2022). *NumPy: the absolute basics for beginners*. Retrieved 10 25, 25, from <http://jalammar.github.io/>
- Albenis Pérez-Alarcón, J. C.-A. (2021). *Alarconpy: a Python Package for Meteorologists*.

Empowering Scientific Computing and Data Manipulation

Harris, C. M. (2020). Array Programming with NumPy. *Nature*, 357–362.

Java Tutorial Developers. (2022). Retrieved from NumPy Tutorial: <https://www.javatpoint.com/>

Johansson, R. (2019). *1. Scientific Computing and Data Science Applications with Numpy, SciPy, and Matplotlib* (2nd ed.). Urayasu-shi.

McKinney, W. (2012). NumPy Basics: Arrays and Vectorized Computation. In W. McKinney, *Python for Data Analysis*. O'Reilly Media, Inc.

Millman, K. J., & Aivazis, M. (2011). Python for Scientists and Engineers. *Computing in Science & Engineering*, 13(2), 9–12. doi:10.1109/MCSE.2011.36

Mulani, S. (2022). *Python Advanced*. Retrieved from Vectors in Python: <https://www.digitalocean.com/community/tutorials/vectors-in-python>

NumPy Developers. (2008-2022). Retrieved from Numpy User Guide: <https://numpy.org/doc/stable/user/whatisnumpy.html>

Oliphant, T. E. (2006). *Guide to NumPy*. Academic Press.

Stefan van der Walt, S. C. (2021). The NumPy Array: A Structure for Efficient Numerical Computation. *IEEE*, 13(2), 22-30.